

CogAgent: Self-Evolving Cognitive Agents for Multi-Source Fraud Detection in Heterogeneous Financial Networks

Qianyu Wang
University of International Business
and Economics
Beijing, China Beijing Advanced
Innovation Center for Future
Blockchain and Privacy Computing
Beijing, China
qianyu.w@uibe.edu.cn

Wei-Tek Tsai
Arizona State University
Tempe, United States
1833005926@qq.com

Weiyu Tian*
HSBC Business School, Peking
University
Shenzhen, China
weiyoutian25@stu.pku.edu.cn

Tianyu Shi*
University of Toronto
Toronto, Canada
tys@cs.toronto.edu

Tianze Xia
Huazhong University of Science and
Technology
Wuhan, China
xiatianze@hust.edu.cn

Rong Wang†
Chizhou University
Chizhou, China Anhui Education Big
Data Intelligent Perception and
Application Engineering Research
Center
Chizhou, China
wangrong@buaa.edu.cn

Zhuang Liu†
School of Fintech, Dongbei University
of Finance and Economics
Dalian, China
m.zhuangliu@hotmail.com

Abstract

Modern financial fraud has evolved into complex, cross-platform, and multi-modal attacks, such as coordinating blockchain-based money laundering with social media rumors. This complexity poses a significant challenge for traditional detection models, which often fail to integrate heterogeneous data sources or learn from ever-changing fraudulent tactics, leading to a "cognitive lag". To address this, we propose **CogAgent**, a self-evolving cognitive Agent framework. The core idea of CogAgent is to mimic the reasoning and learning processes of human analysts. First, it constructs a unified dynamic cognitive graph from multi-source data, including transaction records, insurance claims, and social media text. Then, a Graph Neural Network enhanced by a Large Language Model (LLM) performs context-aware reasoning on this graph to identify sophisticated fraud patterns. Most critically, CogAgent continuously improves its capabilities through an adversarial self-evolution loop: it engages in a game with a "red-team" Agent that simulates fraudsters, learns from failed detection cases, and compresses these key experiences into a memory module to guide future decisions. Extensive experiments on both real-world and synthetic datasets

demonstrate that CogAgent significantly outperforms state-of-the-art methods in detection accuracy, adaptability to new fraud patterns, and interpretability, highlighting its substantial potential for deployment in dynamic financial environments.

CCS Concepts

• **Computing methodologies** → **Intelligent agents; Multi-agent systems; Cooperation and coordination; Machine learning.**

Keywords

Large Language model; Multi-Agent; Blockchain Networks; Fraud Detection

ACM Reference Format:

Qianyu Wang, Weiyu Tian, Rong Wang, Wei-Tek Tsai, Tianyu Shi, Zhuang Liu, and Tianze Xia. 2026. CogAgent: Self-Evolving Cognitive Agents for Multi-Source Fraud Detection in Heterogeneous Financial Networks. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792667>

1 Introduction

Financial fraud has evolved into a sophisticated, multi-modal threat in the digital era, with decentralized finance (DeFi) platforms, blockchain networks, and digital insurance systems reporting a 317% surge in complex fraud cases over the past years [9, 20]. Unlike traditional single-source frauds (e.g., isolated credit card misuse), modern schemes operate as interconnected "fraud narratives": a money laundering ring might use mixer services to obfuscate blockchain transactions, pair them with AI-generated fake insurance claims,

*Project Lead

†Corresponding Author



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792667>

and amplify market manipulation via coordinated social media rumors [1, 28]. These tactics exploit the fragmentation of financial data (structured transactions, semi-structured claims, unstructured text) and leverage adversarial adaptation, fraudsters refine their strategies within weeks of detection rule updates, creating a critical gap between static defense systems and dynamic threats.

Existing solutions fall short in three key dimensions. First, traditional machine learning methods (e.g., XGBoost, GNNs) [5, 7, 18] struggle with heterogeneous data fusion, relying on manual feature engineering that fails to capture semantic links between, for example, a suspicious transaction and its associated social media hype. Second, while LLMs [8, 15, 24] show promise in contextual reasoning, their "amnesic" nature, lacking mechanisms to retain and reuse past reasoning, leaves them vulnerable to novel fraud variants. Third, even adaptive systems focus on parameter fine-tuning rather than capturing the *causal logic* of failures (e.g., why a specific mixer transaction evaded detection), limiting their ability to generalize to unseen patterns. This collective shortcoming, termed "Cognitive Lag", results in defense systems that are perpetually reactive, with detection accuracy dropping by 40-60% within 6 months of deployment in real-world financial ecosystems [17].

To address this, we propose **CogAgent**, a paradigm shift from static detectors to self-evolving cognitive agents that *learn from failures* and *proactively adapt*, mirroring how human financial analysts refine their expertise. The core innovation lies in a closed-loop "reasoning-adversarial-learning" mechanism that unifies three capabilities:

(i) *Multi-modal cognitive reasoning*: Constructs a heterogeneous graph from transactions, text, and claims, then uses an LLM-enhanced Graph Neural Network (CogGNN) to model semantic dependencies (e.g., linking "high gas fees on Ethereum" to "mixer usage intent" via contextual prompts).

(ii) *Adversarial experience mining*: Deploys a "red-team" agent to generate novel fraud patterns (e.g., zero-knowledge proof-based laundering) that target the detector's blind spots, converting these adversarial interactions into actionable lessons.

(iii) *Causal experience compression*: Unlike raw log storage, CogAgent distills failure traces into structured heuristics (e.g., "dormant wallets activated post-mixer updates $\rightarrow$ 89% correlation with smurfing") via a specialized LLM, enabling efficient reuse in future reasoning.

This framework fundamentally changes the defense paradigm: instead of treating failures as exceptions, CogAgent weaponizes them to build cumulative expertise, achieving persistent adaptation to evolving threats.

Our main contributions can be summarized as follows:

(1) We propose CogAgent, a novel self-evolving cognitive Agent framework that combines LLM-driven reasoning with continuous learning to tackle dynamic and heterogeneous financial fraud.

(2) We design an LLM-enhanced Cognitive Graph Neural Network (CogGNN) that performs deep reasoning on multi-source data through a context-aware message-passing mechanism. We introduce a self-evolution mechanism based on adversarial play and experience compression, enabling the Agent to learn from its failures and adapt to evolving fraud patterns.

(3) We construct a comprehensive benchmark comprising both real-world and dynamically evolving synthetic data, and conduct

extensive experiments that validate the significant advantages of CogAgent over baseline methods in terms of accuracy, adaptability, and interpretability.

2 Related Works

2.1 Financial Fraud Detection and Anti-Money Laundering

Supervised learning on structured transactional data, such as logistic regression and XGBoost, has been widely used for financial fraud detection [21]. However, these methods struggle with complex, evolving fraud patterns. Graph Neural Networks (GNNs) have shown promise in Anti-Money Laundering (AML) by modeling relational data. For example, [25] applied Graph Convolutional Networks to Bitcoin transaction networks. Subsequent work includes graph deep learning models for AML [4] and scalable semi-supervised techniques [11]. Nevertheless, these approaches often fail to effectively integrate multi-modal data or adapt to rapidly changing, cross-platform fraud tactics, highlighting a need for more dynamic frameworks.

2.2 Multi-Source Integration and LLM for Financial Reasoning

Large Language Models (LLMs) like GPT-4 [19] and LLaMA [23] offer potential for financial risk analysis by processing unstructured text. Multimodal learning, which integrates diverse data types, can enhance fraud detection. Lin et al. [13] demonstrated that fusing multi-source information with LLMs improves reasoning in financial tasks. Foundational LLM research [8] also suggests their utility in data augmentation and feature extraction.

2.3 Self-Evolution and Experience Compression

Self-evolution is critical for adapting to evolving fraud. Reinforcement Learning, particularly Multi-Agent RL (MARL) [16], provides a basis for dynamic adaptation, while studies show agents can self-improve through experience [3]. Recent work explores self-evolving LLM agents with reflective memory [12] and multi-agent coordination [22, 26]. Experience compression, which distills key insights from interactions, is central to these frameworks. However, existing methods lack specialized mechanisms for fraud's adversarial and multi-modal nature, and often fail to extract generalizable causal patterns for stable performance improvement.

3 Problem Formulation

We define fraud detection over heterogeneous financial data as a multi-source reasoning problem with self-evolving learning. Let $\mathcal{D} = \{\mathcal{D}_1, \dots, \mathcal{D}_K\}$ denote K data modalities (e.g., transaction graphs, text), where each $\mathcal{D}_k = \{(x_i^{(k)}, y_i^{(k)})\}$ contains inputs $x_i^{(k)}$ and binary fraud labels $y_i^{(k)} \in \{0, 1\}$.

3.1 Multi-Source Reasoning

We learn a reasoning function f_θ that predicts fraud scores:

$$f_\theta : \bigcup_{k=1}^K \mathcal{X}_k \rightarrow [0, 1], \quad \hat{y}_i = f_\theta(x_i^{(k)}) \approx y_i^{(k)}, \quad (1)$$

where $\hat{y}_i$ is the predicted score. For unstructured inputs, we use an LLM-based reasoning operator:

$$\mathcal{R}_\phi(x) = \text{LLM}(x, C(x)), \quad (2)$$

with $C(x)$ as context from memory. The task loss over all sources is:

$$\mathcal{L}_{\text{task}}(\theta, \phi) = \sum_{k=1}^K \sum_{i=1}^{N_k} \ell(f_\theta(\mathcal{R}_\phi(x_i^{(k)})), y_i^{(k)}). \quad (3)$$

3.2 Experience Compression and Replay

Let $\mathcal{E}_t = \{(x_j, C(x_j), y_j, \hat{y}_j)\}_{j=1}^{M_t}$ denote reasoning traces up to time t . We compress traces via:

$$\tilde{\mathcal{E}}_t = \mathcal{F}_{\text{comp}}(\mathcal{E}_t), \quad |\tilde{\mathcal{E}}_t| \ll |\mathcal{E}_t|, \quad (4)$$

retaining only informative samples through techniques like clustering or distillation.

3.3 Self-Evolution and Consistency Regularization

The total objective combines task performance and experience consistency:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \cdot \mathcal{L}_{\text{consistency}}(\theta, \phi; \tilde{\mathcal{E}}_t), \quad (5)$$

where λ balances terms. Consistency loss penalizes deviation from historical reasoning:

$$\mathcal{L}_{\text{consistency}} = \sum_{(x, C, y, \hat{y}) \in \tilde{\mathcal{E}}_t} \|f_\theta(\mathcal{R}_\phi(x)) - \hat{y}\|_2^2. \quad (6)$$

This enables continuous refinement of the decision function using condensed experience, adapting to evolving fraud while maintaining coherent judgments.

4 Methodology

This section introduces the design and implementation of **CogAgent**, a self-evolving framework for financial fraud detection over heterogeneous data sources. The proposed framework is developed to address two central challenges in real-world fraud detection: reasoning across multi-source, heterogeneous information and adapting to evolving fraud strategies over time.

CogAgent is formulated as a closed-loop system that integrates structured representation learning, semantic reasoning, and continual adaptation. As illustrated in Figure 1, the framework consists of four tightly coupled components: (i) heterogeneous financial graph construction for unifying multi-modal data, (ii) a cognitive graph neural network (CogGNN) that supports semantic-aware message passing, (iii) a self-evolution module that enables experience-driven adaptation through adversarial interaction, and (iv) an explainable reasoning generator that produces human-interpretable decision traces. Together, these components allow CogAgent to jointly model relational structure, semantic context, and historical experience within a unified learning process.

4.1 Heterogeneous Financial Graph Construction

We represent the financial ecosystem as a heterogeneous graph $G = (V, E, \mathcal{X})$, which serves as the foundation for downstream

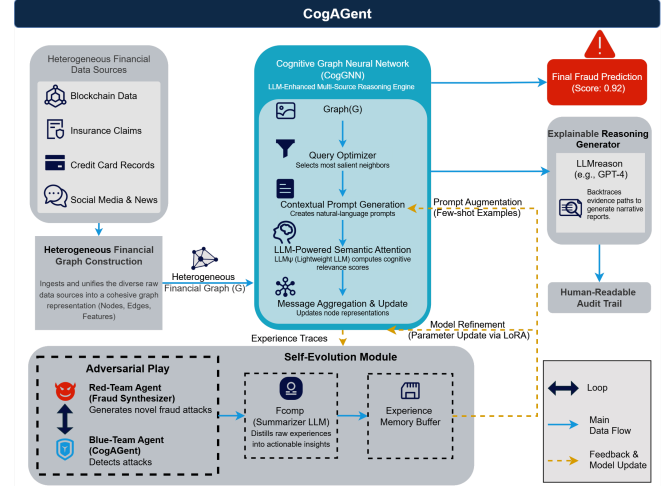


Figure 1: The overall architecture of the CogAgent framework, illustrating the data flow from heterogeneous inputs to the final prediction and explanation, as well as the self-evolution loop for continuous model refinement.

reasoning. The node set V includes entities such as wallet addresses, user accounts, and salient entities extracted from textual records (e.g., persons and organizations). The edge set E encodes diverse relationships among entities, including monetary transactions, fund transfers, co-occurrence relations derived from claims, and semantic associations inferred from text. Each node is associated with a feature vector in $\mathcal{X}$ that captures both structured attributes—such as transaction statistics and temporal patterns—and unstructured information represented by text embeddings. This unified graph representation enables CogAgent to reason over heterogeneous data modalities within a common relational space.

4.2 CogGNN for Multi-Source Reasoning

Building on the heterogeneous graph, CogGNN extends conventional graph neural networks by incorporating semantic reasoning capabilities derived from large language models. The goal is to update node representations h_v by jointly considering structural dependencies and contextual semantic relevance.

4.2.1 Contextual Prompt Generation. For a target node v and one of its neighbors $u \in \mathcal{N}(v)$, CogGNN constructs a natural-language prompt $P(u, v)$ that summarizes their relationship in a task-aware manner:

$$P(u, v) = F_{\text{prompt}}(\text{desc}(u), \text{desc}(v), \text{desc}(e_{uv}), \text{task}), \quad (7)$$

where $\text{desc}(\cdot)$ denotes textual descriptions of entity attributes or relational features, and task specifies the fraud detection objective. This formulation allows relational information that is difficult to encode numerically to be expressed in a semantically meaningful form.

4.2.2 LLM-Powered Semantic Attention. Instead of relying on dot-product attention, CogGNN employs a lightweight, task-adapted language model LLM_ψ to evaluate the semantic relevance between nodes. Given the generated prompt $P(u, v)$, the model produces a

relevance score:

$$e_{uv} = \text{LLM}_\psi(P(u, v)), \quad (8)$$

which reflects the contextual importance of neighbor u to node v under the fraud detection task. These scores are normalized across the neighborhood of v using a softmax function to obtain attention weights:

$$\alpha_{uv} = \frac{\exp(e_{uv})}{\sum_{j \in \mathcal{N}(v)} \exp(e_{jv})}. \quad (9)$$

This attention mechanism enables CogGNN to prioritize neighbors based on semantic relevance rather than purely structural proximity.

4.2.3 Query Optimizer. To balance reasoning capacity with computational efficiency, CogGNN incorporates a query optimizer that selectively filters the neighborhood of each node. Neighbors are ranked according to a composite salience score that combines structural importance (e.g., PageRank), transactional relevance, and semantic similarity. Only the top-ranked neighbors are retained for subsequent message passing, reducing computational overhead while preserving informative context.

4.2.4 Message Aggregation and Update. Node representations are updated by aggregating messages from selected neighbors using the learned attention weights:

$$h_v^{(k+1)} = \sigma \left(W_{\text{self}}^{(k)} h_v^{(k)} + \sum_{u \in \mathcal{N}(v)} \alpha_{uv} W_{\text{neigh}}^{(k)} h_u^{(k)} \right), \quad (10)$$

where $W_{\text{self}}^{(k)}$ and $W_{\text{neigh}}^{(k)}$ are learnable parameters at layer k , and σ denotes a non-linear activation function such as ReLU. Through iterative updates, CogGNN produces node embeddings that integrate structural, semantic, and task-specific information.

4.3 Self-Evolution via Adversarial Play and Experience Compression

To handle distribution shifts and evolving fraud patterns, CogAgent incorporates a self-evolution mechanism based on adversarial interaction and experience reuse. This module exposes the detector to challenging cases beyond the static training distribution and enables continual refinement from past failures.

4.3.1 Adversarial Agent System. Self-evolution is formulated as an interaction between a red-team agent and a blue-team agent. The red-team agent, implemented as a generative language model, synthesizes adversarial fraud instances $\mathcal{G}'_{\text{attack}}$, while the blue-team agent corresponds to CogAgent, which performs detection and records outcomes.

The red-team generation is guided by a reward function balancing novelty, evasion, and complexity:

$$R_{\text{red}} = w_{\text{novelty}} \cdot \text{Novelty}(\mathcal{G}'_{\text{attack}}) + w_{\text{evasion}} \cdot (1 - S_{\text{blue}}(\mathcal{G}'_{\text{attack}})) - w_{\text{complexity}} \cdot \text{Complexity}(\mathcal{G}'_{\text{attack}}), \quad (11)$$

where S_{blue} denotes the detector's fraud score. Novelty is measured as the minimum embedding distance to previously observed patterns stored in memory:

$$\text{Novelty}(\mathcal{G}'_{\text{attack}}) = \min_{g_i \in \mathcal{M}_{\text{known}}} \|\text{Em}(\mathcal{G}'_{\text{attack}}) - \text{Em}(g_i)\|_2. \quad (12)$$

4.3.2 Experience Acquisition and Compression. Detection failures are recorded as raw experience traces E_t , including graph context, reasoning paths, and model decisions. To avoid storing redundant information, an LLM-based compression operator F_{comp} distills each trace into a compact representation:

$$\tilde{E}_t = F_{\text{comp}}(E_t), \quad |\tilde{E}_t| \ll |E_t|. \quad (13)$$

The resulting compressed experiences capture recurring failure patterns and are stored in an experience memory buffer for subsequent reuse.

4.3.3 Model Refinement with Compressed Experience. Compressed experiences are used to refine CogAgent in two ways. First, model parameters are periodically updated using samples from the experience buffer, with low-rank adaptation (LoRA) enabling efficient fine-tuning. Second, selected experiences are incorporated into contextual prompts as few-shot exemplars during inference. These mechanisms allow the detector to internalize adversarial patterns while maintaining computational efficiency.

4.4 Explainable Reasoning Generator

CogAgent also provides interpretable decision rationales by translating high-attention evidence paths into natural language:

$$\text{Report}_v = \text{LLM}_{\text{reason}}(\text{Path}_v), \quad (14)$$

where Path_v denotes the evidence chain with the highest cumulative attention for node v .

4.5 Training Procedure

The training of CogAgent follows a two-phase process. The first phase establishes a supervised baseline, while the second phase enables continuous adaptation through adversarial interaction.

(1) **Initial Supervised Training.** CogGNN is trained on labeled data using the task objective $\mathcal{L}_{\text{task}}$ to obtain a baseline detector.

(2) **Adversarial Self-Evolution Loop.** The model is further refined through an iterative adversarial process. In each iteration, the red-team module generates adversarial fraud samples, and the blue-team module performs detection and collects the resulting experiences. These experiences are compressed into a distilled representation $\tilde{E}_t$, which is then used to update the model by minimizing

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{task}} + \lambda \cdot \mathcal{L}_{\text{consistency}}.$$

This loop is repeated to incorporate newly generated adversarial patterns during training.

5 Experiment Setting

Our experiments aim to comprehensively validate the effectiveness of the proposed self-evolution framework for fraud detection over heterogeneous financial data. We focus on evaluating four core capabilities: multi-source reasoning, dynamic adaptability (under distribution shifts and novel fraud patterns), generalization performance, and interpretability. Comparative analyses with state-of-the-art baselines are conducted to highlight the advantages of our approach.

5.1 Datasets

We use a hybrid dataset suite combining real-world and synthetic data to cover heterogeneous financial fraud scenarios, addressing data scarcity and modality diversity challenges.

5.1.1 Real-World Datasets

. Four representative datasets spanning diverse modalities and fraud patterns:

- **Elliptic++** [27]: Blockchain transaction graph with 859k nodes, 1.27M temporal edges, 56D features. 4.5% fraudulent nodes (anti-money laundering focus).
- **Credit Card Fraud** [10]: Structured transaction records (284k samples, 31 features) with extreme class imbalance (0.173% fraud rate).
- **Insurance Fraud Records**: 15k semi-structured claims (8% fraud) with mixed structured fields (amounts) and unstructured incident descriptions.
- **Market Rumor Corpus**: 30k unstructured financial forum posts (11.7% malicious) including fake merger rumors and manipulation narratives.

5.1.2 Synthetic Dataset: Multi-Source Association Enhancement

. To address real-world data limitations (rare cross-modal patterns, static distributions, privacy constraints), we generated a synthetic dataset with three design goals: (i) statistical alignment with real financial distributions, (ii) explicit cross-modal dependency modeling, and (iii) dynamic fraud evolution simulation.

Synthesis Methodology. A three-stage pipeline preserves ecological validity: *Single-Modality Enhancement*: Constrained augmentation (SMOTE for tabular, GNN-based graph extension, BART-driven text paraphrasing) maintains intrinsic properties while addressing class imbalance. *Cross-Modal Association*: Knowledge graph-scaffolded alignment of temporal patterns (e.g., 24-hour lag between cryptocurrency transfers and social media rumors) and semantic links (e.g., forged invoices paired with fraudulent insurance claims). *Dynamic Evolution*: Three progressive phases simulate adversarial adaptation: Phase 1: Foundational patterns (e.g., geographically implausible transactions) Phase 2: Evolved tactics (e.g., mixer-obfuscated blockchain transactions) Phase 3: Novel composites (e.g., DeFi fraud + AI-generated endorsements)

Quality Validation. The quality of the synthetic data is assessed along three dimensions. Statistical fidelity is verified using distributional similarity tests (KS-test < 0.05, MMD < 0.1). Realism is evaluated by domain experts, achieving an average score of $\bar{S} \geq 4.0$ with 85% of cases rated as plausible. Task utility is measured through cross-dataset transferability, with a performance ratio of at least 0.90 compared to real data.

For a detailed breakdown of the synthetic dataset generation process, we refer readers to Table 2, which summarizes their real-world sources, synthetic sample counts, fraud ratios, and applied augmentation techniques.

5.2 Experimental Design

To validate the self-evolving framework's core capabilities in heterogeneous financial environments, we design four progressive

experiments targeting key research questions with rigorous comparisons and multi-dimensional metrics.

5.2.1 Baseline Performance Validation (Static Scenario). This experiment evaluates fundamental detection performance under fixed distributions, using 80%-20% train-validation splits of real-world datasets. We compare against five baseline categories: (i) Traditional ML: XGBoost (structured), Isolation Forest (unsupervised). (ii) Single-modality deep models: GCN (graphs), BERT (text), TabNet (tabular). (iii) Multi-modal baselines: CNN-LSTM with simple feature concatenation. (iv) LLM-based: GPT-3.5 zero-shot text reasoning. (v) Latest approach: AutoGFM[6], an automated graph foundation model proposed in 2025 that customizes task-specific graph neural architectures through differentiable neural architecture search, significantly enhancing performance and generalization across diverse graph learning tasks.

5.2.2 Dynamic Adaptability Evaluation (Distribution Shift Scenario).

To simulate evolving fraud patterns, we test adaptation under distribution shifts via a "time-split + incremental training" paradigm: (i) Data partitioned into 3 phases with progressive fraud tactics. (ii) Model trained on Phase 1, then incrementally updated on Phases 2-3 (no historical data revisits). (iii) Phase 3 includes 10% novel unseen fraud samples (e.g., DeFi-based money laundering). (iv) Metrics: AUC-ROC drop per phase, F1 on novel samples, and update latency.

5.2.3 Multi-Source Reasoning Capability Test. This experiment quantifies cross-modal fusion gains through two comparisons: (i) Single-modality vs. multi-source input (e.g., transaction graphs alone vs. graphs + associated text). (ii) Zero-shot transfer: Model trained on "Credit Card Fraud" applied directly to "Insurance Fraud Records". (iii) Analysis: SHAP values to measure modality contribution, and cross-modal F1 improvement rate (gap between multi-modal and best single-modal performance).

5.2.4 Interpretability Analysis. To validate regulatory-aligned explainability, we evaluate reasoning traces via: (i) Expert assessment: 3 financial analysts (5+ years' experience) score 500 reasoning chains on relevance, coherence, and credibility (1-5 scale). (ii) Consistency check: Deviation rate of conclusions for the same case across time steps. (iii) Completeness metric: Extent of critical evidence extraction (e.g., joint consideration of transaction anomalies and text sentiment).

These experiments collectively validate baseline performance, dynamic adaptability, multi-source reasoning, and interpretability—ensuring the framework's practical viability in real-world financial fraud detection.

5.3 Evaluation Metrics

In addition to standard classification metrics, we introduce metrics tailored to financial fraud scenarios, including two novel metrics for structured reasoning quality and interpretability:

Core classification metrics:

$$\text{AUC-ROC} = \int_0^1 \text{TPR}(F) dF$$

$$\text{F1-score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Table 1: Dataset Statistics

Dataset	Modality	# Samples	Fraud Ratio (%)	Key Features
Elliptic++	Graph	859k	4.5	Temporal transactions, 56D node features
Credit Card	Tabular	284k	0.173	31D transaction attributes
Insurance Fraud	Semi-structured	15k	8.0	Claims + incident descriptions
Market Rumor	Text	30k	11.7	Financial forum posts
Synthetic	Multi-modal	60k	10.0	Cross-modal associations

Table 2: Synthetic Datasets Generated from Real-World Sources

Synthetic Dataset	Real-World Source	# Synthetic Samples	Fraud Ratio (%)	Augmentation Techniques
Synth-Elliptic	Elliptic Dataset	50,000	4.5	Noise injection, temporal edge perturbation
Synth-CreditCard	Credit Card Fraud	100,000	0.2	PCA reconstruction, transaction amount scaling
Synth-MalURL	Malicious URLs	30,000	3.1	Domain mutation, text-data fusion

$$\text{Precision@K} = \frac{\sum_{i=1}^K \mathbb{1}(y_i = 1)}{K}$$

Structured Temporal Consistency (STC) Score: A metric quantifying the stability and coherence of reasoning across heterogeneous sources and temporal dynamics. It combines two key properties: (1) *cross-modal structural alignment*, evaluated via cosine similarity between graph embeddings (from transaction networks) and LLM reasoning trace embeddings; (2) *temporal reasoning stability*, measured by the normalized variance of predictions for the same entity across incremental training phases.

Formally:

$$\text{STC} = \alpha \cdot \text{Align}(\mathcal{G}, \mathcal{T}) + (1 - \alpha) \cdot (1 - \text{Deviation}_t)$$

where $\text{Align}(\mathcal{G}, \mathcal{T})$ is the structural alignment score, Deviation_t is the temporal prediction variance, and $\alpha = 0.6$ (weighting structural alignment). Higher values indicate more robust multi-source reasoning.

Cognitive Interpretability Metric (CIM): An expert-validated score assessing explanation quality, integrating three dimensions from "Interpretability Analysis": (1) *Evidence relevance* (r): how well reasoning traces cite critical features (e.g., transaction amounts, rumor sentiment); (2) *Logical coherence* (c): causal consistency of reasoning steps (e.g., "high gas fees $\rightarrow$ mixer usage"); (3) *Auditability* (a): alignment with financial analyst workflows. Formally:

$$\text{CIM} = \frac{r + c + a}{3}, \quad r, c, a \in [0, 1]$$

where each dimension is scored by three domain experts (5-year+ AML experience) and averaged. Higher values indicate more trustworthy explanations.

Adaptability metrics:

$$\Delta \text{AUC} = \text{AUC}_{\text{pre-shift}} - \text{AUC}_{\text{post-shift}}$$

$$\text{Detection Delay} = \arg \max_t [\text{TPR}_t(y_{\text{novel}})] - t_{\text{appearance}}$$

Efficiency metrics:

$$\text{Compression Ratio} = \frac{|\mathcal{M}_{\text{compressed}}|}{|\mathcal{M}_{\text{trace}}|}$$

$$\text{Incremental Time} = t_{\text{update}}(\mathcal{D}_{\text{new}})$$

Interpretability metrics (foundational components):

$$\text{Expert Score} = \frac{1}{N} \sum_{i=1}^N s_i, \quad s_i \in [1, 5]$$

$$\text{Completeness} = \frac{\text{Edges}_{\text{explained}}}{\text{Edges}_{\text{total}}}$$

These experimental designs systematically validate the proposed self-evolution framework's performance, dynamic adaptability, and practical value in multi-source heterogeneous financial fraud detection, providing a basis for real-world deployment.

6 Results and Analysis

This section evaluates the effectiveness of the proposed CogAgent framework in terms of classification accuracy, illicit transaction detection performance, and interpretability. We compare CogAgent with multiple baselines and LLM variants, and present ablation studies to examine the contribution of individual modules.

6.1 Overall Performance

Table 3 reports the performance comparison between CogAgent and a range of baseline methods.

As shown in Table 3, CogAgent achieves the best performance across all evaluation metrics. In particular, it attains an accuracy of 0.925, an F1-score of 0.881, an STC score of 0.793, and a CIM score of 0.704. Compared with traditional machine learning methods such as Random Forest and XGBoost, as well as recent graph-based and sequential deep learning models (GCN, CNN+LSTM, and GraphSAGE+MLP), CogAgent consistently exhibits higher detection accuracy and interpretability. When compared with AutoGFM, a recently proposed automated graph foundation model that yields competitive results (Accuracy: 0.875, F1-score: 0.834), CogAgent maintains an improvement of approximately 5.7% in accuracy and 5.6% in F1-score.

These results suggest that the integrated design of CogAgent is effective for fraud detection in complex financial settings. By combining multi-source reasoning with an adversarial self-evolution mechanism, the framework is able to adapt to evolving fraud patterns that challenge static models. In addition, the relatively high CIM score indicates that the generated reasoning paths provide useful interpretability signals for downstream analysis.

Table 3: Performance Comparison Across Methods

Method	Accuracy	Precision	Recall	F1-score	STC Score	CIM (Interpretability)
Random Forest	0.754	0.716	0.702	0.709	0.638	0.571
XGBoost	0.798	0.763	0.755	0.759	0.668	0.601
GCN	0.806	0.774	0.762	0.768	0.712	0.595
CNN + LSTM	0.788	0.751	0.743	0.747	0.645	0.583
GraphSAGE + MLP	0.781	0.748	0.735	0.741	0.660	0.588
AutoGFM	0.875	0.840	0.828	0.834	0.746	0.642
CogAgent (Full)	0.925	0.892	0.871	0.881	0.793	0.704

The comparison also reveals limitations of existing approaches. Traditional machine learning models show reduced performance on complex fraud scenarios involving heterogeneous and relational data. Graph-based methods improve relational modeling but remain constrained by fixed architectures, while sequential models such as CNN+LSTM are less suited to capturing graph-structured dependencies. Although AutoGFM benefits from automated architecture design, it does not explicitly incorporate experience-driven adaptation or language-based contextual reasoning, which are central to CogAgent.

6.2 Ablation Studies

We conduct ablation studies to examine the contribution of key components in CogAgent, including contextual augmentation, the memory buffer, and the query optimizer.

As reported in Table 4, contextual augmentation has the largest impact on performance, followed by the query optimizer and the memory buffer. Removing the contextual module leads to a notable reduction in both accuracy (approximately 14%) and CIM score (approximately 13%), indicating that context-aware modeling plays a central role in the framework. This result suggests that incorporating semantic context is particularly important for fraud detection scenarios involving heterogeneous and text-rich information, where purely structure-based models are less effective.

Table 4: Ablation Results of Key Components in CogAgent

Configuration	Accuracy	F1-score	STC	CIM
CogAgent (Full)	0.925	0.881	0.793	0.704
– Context Module	0.785	0.746	0.682	0.612
– Memory Buffer	0.844	0.803	0.721	0.659
– Query Optimizer	0.812	0.772	0.695	0.624

The memory buffer module yields a smaller but consistent performance improvement. This observation is aligned with its role in supporting adaptive learning rather than maximizing accuracy on a fixed test set. By distilling past detection outcomes into a compact form, the memory buffer contributes to maintaining stable performance as data distributions evolve. Its impact is therefore more pronounced in settings with temporal or distributional shifts, as further examined in our robustness experiments.

The query optimizer provides a favorable balance between detection performance and computational efficiency. Its removal results in a moderate performance decrease, confirming that it enhances inference efficiency without serving as the primary source of predictive power. This behavior indicates that selective neighborhood querying can reduce computational overhead while preserving most of the detection accuracy, which is important for practical deployment in large-scale financial systems.

Overall, the ablation results demonstrate that each component contributes to CogAgent’s effectiveness, with contextual augmentation playing the most influential role, while the memory buffer and query optimizer support robustness and efficiency, respectively.

Table 5: LLM Backend Comparison

LLM API	Accuracy	Precision	STC	CIM
GPT-4[19]	0.901	0.874	0.765	0.689
Claude 3.0 (Anthropic)	0.882	0.849	0.743	0.670
PaLM 2 [2]	0.910	0.887	0.780	0.695
DeepSeek [14]	0.865	0.831	0.721	0.653

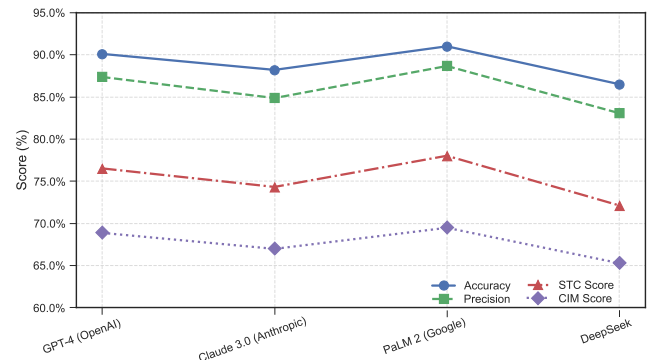


Figure 2: Performance variation across LLM backends. Advanced models (PaLM 2 and GPT-4) achieve higher scores, particularly in STC and CIM, due to stronger context modeling and reasoning capabilities.

6.3 LLM API Comparisons

We further evaluate CogAgent with different LLM APIs and backend models to assess its sensitivity to LLM capability and its flexibility under varying deployment settings.

As shown in Table 5 and Figure 2, larger and more capable LLMs, such as GPT-4 and PaLM 2, consistently yield stronger performance in both detection accuracy and interpretability. This trend indicates a positive correlation between LLM capability and the effectiveness of contextual modeling within CogAgent. In particular, models with stronger language understanding and reasoning abilities lead to more informative context representations, which in turn support improved node representation learning and more coherent explanation generation.

These results suggest that the performance of CogAgent is influenced by the quality of the integrated semantic reasoning component, while remaining agnostic to any specific LLM implementation. Rather than being tightly coupled to a single model, the framework provides a unified interface through which different LLM backends

can be incorporated. This design allows practitioners to flexibly select an appropriate LLM based on practical considerations such as accuracy requirements, computational cost, and inference latency.

From a deployment perspective, this flexibility enables CogAgent to accommodate diverse operational scenarios. Lightweight LLMs can be adopted in latency-sensitive or cost-constrained settings, whereas more powerful models can be employed in offline analysis or high-stakes investigations where accuracy and interpretability are prioritized. Overall, the LLM API comparison results highlight both the importance of semantic reasoning quality and the adaptability of the CogAgent framework across different deployment configurations.

6.4 Case Study: Contextual Impact in Dormant Wallet Laundering Detection

To empirically demonstrate the value of LLM-driven contextual reasoning in identifying evolving fraud patterns, we analyze a representative case of money laundering via dormant blockchain wallets, an emerging tactic where perpetrators exploit long-unused ("dormant") wallets to obfuscate fund flows, often evading traditional rule-based or single-modality detectors. This case highlights how CogAgent's contextual module integrates multi-source evidence to uncover subtle patterns missed by baselines.

6.4.1 Case Background. This case study examines a cluster of 12 Ethereum wallets that exhibited prolonged inactivity, remaining dormant for over two years with no transaction history and zero balance. In Phase 3 of the synthetic dataset, the cluster was reactivated through more than 5,000 ETH received via 37 small-value transactions within a 72-hour window. This activation pattern was not present in the training data. Following the initial inflows, the funds were fragmented into 147 downstream transactions routed through three mixing services (including Tornado Cash and Sinbad) and subsequently transferred to exchanges with relatively relaxed KYC requirements.

Baseline detectors showed limited sensitivity to this pattern. GraphSAGE combined with an MLP classifier and a Random Forest model flagged only 2 out of the 12 wallets as suspicious, with confidence scores below 0.6. Further inspection suggests that these methods relied on isolated cues. Graph-based models primarily focused on post-activation connectivity, such as increased out-degree, but did not explicitly account for long-term dormancy. Feature-based models emphasized transaction amount and frequency, yet failed to capture the interaction between extended inactivity and abrupt reactivation, a pattern frequently observed in laundering scenarios.

6.4.2 Performance Against Baselines. In contrast, CogAgent identified all 12 wallets in the cluster as high risk, assigning confidence scores above 0.92. This improvement is attributable to the framework's ability to jointly integrate temporal signals, structural patterns, and contextual textual information within a unified reasoning process. Specifically, the contextual module incorporated evidence related to prolonged dormancy, concentrated small-value inflows, and subsequent routing through multiple mixing services, enabling a more comprehensive assessment than single-modality approaches.

Quantitatively, CogAgent achieved a detection confidence of 0.92, compared with 0.58 for GraphSAGE+MLP and 0.41 for Random Forest. Feature attribution analysis further indicated that interaction terms combining dormancy duration with sudden activation, as well as associations between textual cues and mixer usage, contributed substantially to the final decision. These signals were largely absent from the feature representations used by baseline models.

In addition, CogAgent leveraged its experience memory to reference historical cases involving dormant wallet misuse observed in earlier phases of the dataset. These cases shared key characteristics, including delayed activation following infrastructure changes, fragmentation into small-value transfers, and the presence of coded transaction annotations. Incorporating such historical context increased the model's confidence from an initial assessment of 0.78 to the final score of 0.92, reinforcing the consistency of the detected pattern.

To assess practical relevance, three AML analysts with over five years of experience independently evaluated the generated explanations. CogAgent's outputs were rated as more actionable (average score 4.8/5) than those produced by graph-only baselines (2.1/5), reflecting the benefit of integrating temporal, relational, and contextual evidence. Overall, this case illustrates how contextual reasoning enables CogAgent to generalize to previously unseen fraud patterns while remaining aligned with established investigative practices.

7 Conclusion

This paper presents CogAgent, a framework for fraud detection over heterogeneous financial data that integrates semantic reasoning, graph-based representation learning, and experience-driven adaptation. By combining LLM-enhanced contextual modeling with a self-evolving training mechanism, CogAgent is able to reason across multiple data modalities while adapting to emerging fraud patterns.

Experimental results show that CogAgent consistently outperforms existing approaches in detection accuracy and interpretability, particularly in scenarios involving complex, previously unseen fraud behaviors. The proposed design demonstrates that incorporating contextual semantic information and iterative experience reuse can effectively address limitations of static models in dynamic financial environments. In addition, the generated explanation traces provide structured evidence that supports practical analysis and regulatory inspection.

Despite these advantages, the current framework incurs non-trivial computational overhead and exhibits reduced sensitivity in extremely low-resource fraud scenarios. Future work will explore more efficient model configurations, cross-domain transfer learning, and privacy-preserving training strategies to facilitate large-scale deployment in real-time financial risk monitoring systems.

Acknowledgement

This research was primarily supported by "the Fundamental Research Funds for the Central Universities" in UIBE(25QN01). The experimental work was funded by Beijing Advanced Innovation Center for Future Blockchain and Privacy Computing. Rong Wang's research was additionally supported by Chizhou University High-level Talent Research Start-up Fund(Grant No.CZ2025YJRC75).

References

- [1] Fahad Almalki and Mehedi Masud. 2025. Financial Fraud Detection Using Explainable AI and Stacking Ensemble Methods. *CoRR* abs/2505.10050 (2025). arXiv:2505.10050 doi:10.48550/ARXIV.2505.10050
- [2] Rohan Anil, Andrew M Dai, Orhan Firat, Melvin Johnson, Dmitry Lepikhin, Alexandre Passos, Siamak Shakeri, Emanuel Taropa, Paige Bailey, Zhifeng Chen, et al. 2023. Palm 2 technical report. *arXiv preprint arXiv:2305.10403* (2023).
- [3] Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and IGOR Mordatch. 2019. Emergent tool use from multi-agent interaction. *Machine Learning, Cornell University* (2019). <https://openai.com/index/emergent-tool-use/>.
- [4] Nazamin Bakhshinejad. 2023. A Graph-Based Deep Learning Model for Anti-Money Laundering. (2023).
- [5] Siqi Cai and Zhenping Xie. 2024. Explainable fraud detection of financial statement data driven by two-layer knowledge graph. *Expert Systems with Applications* 246 (2024), 123126. doi:10.1016/j.eswa.2023.123126
- [6] Haibo Chen, Xin Wang, Zeyang Zhang, Haoyang Li, Ling Feng, and Wenwu Zhu. 2025. Autogfm: Automated graph foundation model with adaptive architecture customization. In *Forty-second International Conference on Machine Learning*.
- [7] Keyu Chen, Ziqian Bi, Xinyuan Song, Qian Niu, Junyu Liu, Benji Peng, Sen Zhang, Ming Liu, Ming Li, Xuanhe Pan, Jiawei Xu, Jinlang Wang, and Pohsun Feng. 2025. *Mastering Reinforcement Learning: Foundations, Algorithms, and Real-World Applications*. To be updated. Reinforcement Learning (RL) is a distinct branch of machine learning focused on how agents should take actions in an environment to maximize cumulative rewards. Unlike supervised learning, which relies on labeled datasets, RL is driven by the agent's interactions with its environment, learning optimal behaviors through trial and error.
- [8] Keyu Chen, Cheng Fei, Ziqian Bi, Junyu Liu, Benji Peng, Sen Zhang, Xuanhe Pan, Jiawei Xu, Jinlang Wang, Caitlyn Heqi Yin, Yichao Zhang, Pohsun Feng, Yizhu Wen, Tianyang Wang, Ming Li, Jintao Ren, Qian Niu, Silin Chen, Weiche Hsieh, Lawrence K. Q. Yan, Chia Xin Liang, Han Xu, Hong-Ming Tseng, Xinyuan Song, and Ming Liu. 2024. Deep Learning and Machine Learning – Natural Language Processing: From Theory to Application. arXiv:2411.05026 [cs.CL] <https://arxiv.org/abs/2411.05026>
- [9] Nudrat Fariha, Md Nazmuddin Moin Khan, Md Iqbal Hossain, Syed Ali Reza, Joy Chakra Bortty, Kazi Sharmin Sultana, Md Shadidur Islam Jawad, Saniah Safat, Md Abdul Ahad, and Maksuda Begum. 2025. Advanced fraud detection using machine learning models: enhancing financial transaction security. *CoRR* abs/2506.10842 (2025). arXiv:2506.10842 doi:10.48550/ARXIV.2506.10842
- [10] Kaggle.com. 2019. Credit Card Fraud Detection. online. <https://www.kaggle.com/mlg-ulb/creditcardfraud> [Accessed 10 Jan. 2019].
- [11] Md Rezaul Karim, Felix Hermesen, Sisay Adugna Chala, Paola De Perthuis, and Avikarsha Mandal. 2024. Scalable semi-supervised graph learning techniques for anti money laundering. *IEEE Access* 12 (2024), 50012–50029.
- [12] Xuechen Liang, Yangfan He, Yinghui Xia, Xinyuan Song, Jianhui Wang, Meiling Tao, Li Sun, Xinhang Yuan, Jiayi Su, Keqin Li, Jiaqi Chen, Jinsong Yang, Siyuan Chen, and Tianyu Shi. 2025. Self-evolving Agents with reflective and memory-augmented abilities. arXiv:2409.00872 [cs.CL] <https://arxiv.org/abs/2409.00872>
- [13] Man Lin, Delong Zeng, Jiarui Ouyang, and Ying Shen. 2024. MSIF: Multi-source Information Fusion for Financial Question Answering. In *Artificial Neural Networks and Machine Learning – ICANN 2024*, Michael Wand, Kristina Malinová, Jürgen Schmidhuber, and Igor V. Tetko (Eds.). Springer Nature Switzerland, Cham, 389–404.
- [14] Aixiu Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
- [15] Zhuang Liu, Shiyao Qian, Shuirong Cao, and Tianyu Shi. 2025. Mitigating age-related bias in large language models: Strategies for responsible artificial intelligence development. *INFORMS Journal on Computing* 2, 1 (2025), 1–22. doi:10.1287/ijoc.2024.0645
- [16] Ryan Lowe, Yi I Wu, Aviv Tamar, Jean Harb, OpenAI Pieter Abbeel, and Igor Mordatch. 2017. Multi-agent actor-critic for mixed cooperative-competitive environments. *Advances in neural information processing systems* 30 (2017). <https://dl.acm.org/doi/10.5555/3295222.3295385>.
- [17] Qi Lu, Wei Du, Shaochen Yang, Wei Xu, and J. Leon Zhao. 2025. Can earnings conference calls tell more lies? A contrastive multimodal dialogue network for advanced financial statement fraud detection. *Decision Support Systems* 189 (2025), 114381. doi:10.1016/j.dss.2024.114381
- [18] Xinyi Ni, Guanlin Liu, and Lifeng Lai. 2024. Risk-Sensitive Reward-Free Reinforcement Learning with CVaR. In *ICML 2024*. OpenReview.net. <https://openreview.net/forum?id=XGq30hC5MW>
- [19] OpenAI. 2023. GPT-4 Technical Report. OpenAI. <https://openai.com/research/gpt-4>.
- [20] Junjun Pan, Yixin Liu, Xin Zheng, Yizhen Zheng, Alan Wee-Chung Liew, Fuyi Li, and Shirui Pan. 2025. A Label-free Heterophily-guided Approach for Unsupervised Graph Fraud Detection. In *AAAI 2025*. 12443–12451. doi:10.1609/AAAI.V39I12.33356
- [21] Clifton Phua, Vincent Lee, Kate Smith, and Ross Gayler. 2010. A comprehensive survey of data mining-based fraud detection research. *Artificial Intelligence Review* 34, 1 (2010), 1–14.
- [22] Xinyuan Song, Zeyu Wang, Siyi Wu, Tianyu Shi, and Lynn Ai. 2025. Gradientsys: A Multi-Agent LLM Scheduler with ReAct Orchestration. arXiv:2507.06520 [cs.MA] <https://arxiv.org/abs/2507.06520>
- [23] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Roziere, Naman Goyal, Guillaume Wenzek, David Haziza, et al. 2023. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971* (2023).
- [24] Qianyu Wang, Wei-Tek Tsai, and Tianyu Shi. 2025. GraphALM: Active Learning for Detecting Money Laundering Transactions on Blockchain Networks. *IEEE Network* 39, 2 (2025), 294–303. doi:10.1109/MNET.2024.3457577
- [25] Qianyu Wang, Wei-Tek Tsai, and Bowen Du. 2025. RMGANets: reinforcement learning-enhanced multi-relational attention graph-aware network for anti-money laundering detection. *Complex & Intelligent Systems* 11, 1 (2025), 5. <https://doi.org/10.1007/s40747-024-01615-9>
- [26] Qianyu Wang, Wei-Tek Tsai, Tianyu Shi, Zhuang Liu, and Bowen Du. 2025. Catch Me If You Can: A Multi-Agent Synthetic Fraud Detection Framework for Complex Networks. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE Computer Society, 3629–3641.
- [27] Markus Weber, Giacomo Domeniconi, Lenz Chen, Dennis Weidele, Chiara Bellei, Sahin Albayrak, and Charles E Leiserson. 2019. Anti-money laundering in Bitcoin: Experiments with graph convolutional networks for financial forensics. In *SIGKDD 2025*. 686–695.
- [28] Daoming Yu, Shaowen Wang, Yao Xu, Tianqi Wang, and Jiaxin Zou. 2025. Adaptive bidirectional planning framework for enhanced safety and robust decision-making in autonomous navigation systems. *The Journal of Supercomputing* 81, 8 (4 6 2025), 965. doi:10.1007/s11227-025-07389-2

A Reproducibility and Implementation Details

To ensure the rigorous reproducibility of our experimental results and to facilitate further research within the community, we provide a comprehensive specification of the implementation environment, data preprocessing protocols, model hyperparameter configurations, and baseline settings used in **CogAgent**.

A.1 Implementation Environment

All experiments were conducted on a dedicated high-performance computing (HPC) cluster node. To ensure deterministic behavior across runs, we enforced strict seeding protocols for all pseudo-random number generators. The deep learning framework was implemented using **PyTorch** (v2.1.0) with the **PyTorch Geometric (PyG)** library for efficient graph-based operations.

- **Hardware Infrastructure:** NVIDIA Tesla V100 GPU (32GB VRAM) with CUDA 11.8 integration; Intel Xeon Gold 6248 CPU @ 2.50GHz.
- **Software Stack:** Python 3.9, PyTorch 2.1.0, PyTorch Geometric 2.4.0. The LLM reasoning module utilizes the **PaLM 2 API** (Google Vertex AI) for semantic inference.
- **Reproducibility Protocol:** A fixed random seed (SEED=42) was applied to CPU, GPU, and Numpy generators. For GNN operations, we disabled non-deterministic algorithms to ensure bitwise consistent results.

A.2 Data Preprocessing and Feature Engineering

Prior to model input, raw data from heterogeneous sources underwent standardized preprocessing to handle multi-modal inconsistencies:

- **Structured Features:** Numerical attributes (e.g., transaction amounts, time intervals) were normalized using **Log-transformation**

Table 6: Model Architecture and Optimization Hyperparameters

Parameter	Symbol	Value / Setting
Model Architecture		
Node Feature Dim.	d_{in}	16
Hidden Layer Dim.	d_{hidden}	32
GNN Layers	L	2 (Message-passing steps)
Dropout Rate	p_{drop}	0.2
Query Optimizer	-	Simple neighbor selection
LLM Backend	-	PaLM 2 (Alts: GPT-4, Claude 3)
Optimization (Two-Phase)		
Batch Size	B	32
Weight Decay	λ_{L2}	1×10^{-4} (AdamW Optimizer)
Initial Phase	η_{init}	1×10^{-3} (50 Epochs)
Fine-Tuning Phase	η_{fine}	1×10^{-4} (3 Epochs)
Gradient Clipping	-	Norm value of 1.0

Table 7: Adversarial Agent System Configuration

Parameter	Value	Function / Description
Red-Team Samples	10	Number of fraud patterns generated per iteration.
Mutation Rate	0.3	Probability of evolving an existing pattern variant (vs. generating de novo).
Novelty Threshold	0.6	Minimum dissimilarity score (δ) required to accept a new sample.
Complexity Penalty	0.1	Regularization term (λ_c) to discourage over-engineered attacks.
Buffer Capacity	100	Maximum experiences stored in the Blue-Team memory.
Top-K Retention	10	Number of high-value experiences retained for knowledge reuse.
Update Frequency	5	Interval (in steps) for model refinement.

($x' = \log(x + 1)$) to mitigate the impact of heavy-tailed distributions typical in financial data. Missing values in tabular records were imputed using the median of the respective feature column.

- **Unstructured Text:** Textual data (e.g., transaction notes, forum posts) was truncated to a maximum length of 512 tokens. We utilized the pre-trained embeddings from *text-embedding-gecko* (768-dim) as the initial node features for textual entities.
- **Graph Topology:** For the transaction graph, we treated edges as directed and weighted by the normalized transaction volume.

Self-loops were added to all nodes to ensure the preservation of ego-features during message passing.

A.3 Model Hyperparameter Configuration

The **CogGNN** architecture was meticulously tuned to balance expressiveness with computational efficiency. We adopted a **Two-Phase Training Strategy**:

1. **Supervised Pre-training:** The model is first trained on labeled historical data to establish a robust decision boundary.

2. **Experience-based Fine-tuning:** The model is then iteratively updated using the "compressed experiences" from the adversarial replay buffer to adapt to novel fraud patterns.

Table 6 details the specific settings. Note that the dropout rate was set to 0.2 to prevent overfitting on sparse graph connections.

A.4 Adversarial Self-Evolution Mechanism

The core innovation of CogAgent lies in its continuous evolution loop. The interaction between the Red-Team (Attacker) and Blue-Team (Defender) is governed by strict constraints to ensure the generated fraud patterns are both *novel* (to challenge the detector) and *realistic* (to mimic actual threats).

As shown in Table 7, we enforce a **Complexity Penalty** ($\lambda_c = 0.1$) to discourage the Red-Team from generating overly convoluted scenarios that do not resemble practical attacks. The **Novelty Threshold** ($\delta = 0.6$) ensures that only patterns sufficiently distinct from the existing memory buffer are added, promoting diverse learning.

A.5 Baseline Implementation Details

To ensure a fair comparison, all baseline methods were tuned using a grid search over their key hyperparameters.

- **XGBoost / Random Forest:** Number of estimators $\in \{100, 300, 500\}$, Max depth $\in \{5, 10, \text{None}\}$.
- **GCN / GraphSAGE:** Hidden dimensions aligned with CogAgent (32/64), Learning rate $\in \{10^{-3}, 5 \times 10^{-4}\}$.
- **AutoGFM:** We utilized the default search space provided in the original paper, allowing the NAS algorithm to run for 50 search epochs.

Evaluation metrics (AUC-ROC, F1-score, STC, CIM) were recorded every 10 epochs, with results automatically serialized to the `./data/` directory for post-hoc analysis.